

SEDS 537 - Machine Learning

Term Project Progress Report

Multilingual Toxic Comment Classification

Student: Yasin Turk - 323011020

Instructor: Prof. Dr. Aytug Onan

Spring 2026

Technical Checkpoint - May 11, 2026

1. Project Title and Problem Definition

Project Title: Multilingual Toxic Comment Classification

This project addresses the task of binary text classification for toxicity detection across multiple languages. Online platforms generate vast amounts of user-generated content in diverse languages, and automated moderation systems must identify toxic comments reliably regardless of the language in which they are written. This is a practically important problem because manual moderation does not scale, and most existing toxicity classifiers are English-centric.

The machine learning task is binary classification: given a comment text, predict whether it is toxic (1) or non-toxic (0). The input is raw comment text in any of several languages (English, Turkish, Spanish, Italian, Portuguese, Russian, French). The output is a toxicity probability score, thresholded for binary prediction.

The dataset used is the Jigsaw Multilingual Toxic Comment Classification dataset from Kaggle. The core challenge is cross-lingual transfer: training data is available only in English, while evaluation is performed on comments in other languages. Additional challenges include class imbalance in toxicity labels and ensuring that parameter-efficient fine-tuning methods (adapters) can capture language-specific toxic patterns effectively.

The project title and scope have not changed since the original proposal.

2. Dataset and Preprocessing Status

Dataset: Jigsaw Multilingual Toxic Comment Classification (Kaggle)

Source: <https://www.kaggle.com/c/jigsaw-multilingual-toxic-comment-classification>

The dataset consists of three files:

- jigsaw-toxic-comment-train.csv: ~223,549 English comments with binary toxicity labels (from the original Jigsaw Toxic Comment Challenge). Used for training.
- validation.csv: ~8,000 comments in 6 non-English languages (Turkish, Spanish, Italian, Portuguese, Russian, French) with binary labels. Used for validation during training.
- test.csv: ~63,812 multilingual comments. Used for final evaluation.

Features: Raw comment text (string). Target: toxic (binary, 0 or 1).

Class distribution: Approximately 9.6% toxic (21,384 samples) and 90.4% non-toxic (202,165 samples) in the training set, indicating notable class imbalance.

Train/validation/test split: The split follows the competition structure. The English training set is used for model training, the multilingual validation set for hyperparameter tuning and early stopping, and the multilingual test set for final evaluation.

Preprocessing steps completed:

- Dataset downloaded and loaded into pandas DataFrames.
- Exploratory data analysis performed: label distribution, text length statistics, language distribution in validation set.
- Text cleaning pipeline implemented: lowercasing, removal of URLs, HTML tags, special characters, and extra whitespace.
- For TF-IDF baselines: tokenization and TF-IDF vectorization (max 50,000 features, unigrams + bigrams).
- For transformer models: tokenization using HuggingFace AutoTokenizer with max_length=128 and padding/truncation.

No critical data access or quality issues have been encountered. The dataset is publicly available on Kaggle and has been fully downloaded.

3. Literature Review Progress

The literature review has focused on three areas: multilingual text classification, toxic comment detection, and parameter-efficient fine-tuning.

For multilingual representation, the foundational work is Devlin et al. (2019) on BERT and its multilingual variant mBERT, which provides cross-lingual transfer by pretraining on 104 languages with a shared WordPiece vocabulary. Conneau et al. (2020) introduced XLM-RoBERTa, which improved cross-lingual performance by training on larger multilingual corpora with better sampling strategies.

For adapter-based fine-tuning, Houlisby et al. (2019) proposed inserting small bottleneck layers (adapters) into transformer blocks, training only these layers while freezing the pretrained weights. Pfeiffer et al. (2020) extended this with AdapterHub and showed that adapter modules are effective for cross-lingual transfer tasks, achieving competitive results with a fraction of the trainable parameters. This is the core approach motivating the proposed method in this project.

For toxic comment detection, the Jigsaw competition series on Kaggle has produced a body of practical work. Top solutions typically employ transformer-based ensembles, data augmentation through translation, and pseudo-labeling. Risch et al. (2020) compared classical ML and deep learning approaches for toxic comment detection and found that BERT-based models significantly outperform TF-IDF-based methods.

The reviewed literature supports the project direction: using mBERT with adapter modules as a parameter-efficient alternative to full fine-tuning for cross-lingual toxicity classification. Classical ML baselines (LR, NB, SVM with TF-IDF) and LSTM serve as reference points to quantify the benefit of pretrained multilingual representations.

4. Baseline Models

Five baseline models have been selected, spanning classical ML, recurrent deep learning, and transformer-based approaches:

1. TF-IDF + Logistic Regression: A simple and interpretable baseline. TF-IDF features (50K, unigrams+bigrams) are fed to a Logistic Regression classifier with L2 regularization. Implementation is complete using scikit-learn. Initial results have been obtained on the English validation split.
2. TF-IDF + Naive Bayes: Multinomial Naive Bayes with TF-IDF features. Used to compare probabilistic classification against discriminative methods under the same feature representation. Implementation is complete.
3. TF-IDF + SVM: Linear SVM with TF-IDF features. Known to perform well on high-dimensional sparse text features and serves as the strongest classical ML baseline. Implementation is complete.
4. LSTM: A bidirectional LSTM with a pretrained multilingual embedding layer (FastText). Captures sequential patterns in text unlike bag-of-words TF-IDF methods. Implementation is in progress; architecture is defined and training script is written, but hyperparameter tuning has not yet been performed.
5. mBERT (full fine-tuning): bert-base-multilingual-cased from HuggingFace, fine-tuned on the English training data with all parameters updated. Serves as the strongest baseline and the reference point for evaluating the proposed adapter approach. Implementation is in progress; training script is written using HuggingFace Trainer API.

The three TF-IDF-based baselines are fully implemented. The LSTM and mBERT baselines have their training pipelines written but require further training and evaluation runs.

5. Initial Experimental Results

Preliminary results have been obtained for the three classical ML baselines on a held-out 20% split of the English training data. The table below summarizes the initial results:

Model	AUC-ROC	F1	Accuracy	Status
-------	---------	----	----------	--------

TF-IDF + Log. Reg.	0.9693	0.6993	0.9527	Implemented
TF-IDF + Naive Bayes	0.9434	0.6759	0.9445	Implemented
TF-IDF + SVM	0.9649	0.7299	0.9555	Implemented
LSTM (BiLSTM)	-	-	-	In progress
mBERT (full fine-tune)	-	-	-	In progress

The initial results show that TF-IDF + Logistic Regression achieves the highest AUC-ROC (0.9693) among classical baselines, followed closely by SVM (0.9649). However, SVM achieves the highest F1-score (0.7299) due to better recall-precision balance. Naive Bayes performs slightly lower as expected (AUC-ROC: 0.9434). The relatively high AUC-ROC but moderate F1 scores reflect the impact of class imbalance, where the models achieve high accuracy by correctly classifying the majority (non-toxic) class but have lower recall on the minority (toxic) class.

Preliminary evaluation on the multilingual validation set (Turkish, Spanish, Italian) shows a significant performance drop: AUC-ROC falls to 0.58-0.63 across all three models, and F1-scores drop below 0.05. This is expected since TF-IDF features are language-specific and cannot transfer across languages. This result confirms the need for multilingual representations such as mBERT and motivates the proposed adapter-based approach.

6. Planned Improvements and Technical Direction

The primary technical direction for the next phase is implementing adapter-based fine-tuning on mBERT. Specifically, the plan is to:

- Insert bottleneck adapter layers into each transformer block of mBERT, following the architecture proposed by Houlsby et al. (2019).
- Freeze all pretrained mBERT parameters and train only the adapter weights and the classification head on the English training data.
- Compare adapter fine-tuning against full fine-tuning in terms of AUC-ROC, F1-score, parameter count, and inference time.
- Experiment with different adapter bottleneck sizes (e.g., 64, 128, 256) to find the optimal capacity-efficiency trade-off.
- Address class imbalance using weighted loss functions or oversampling of the minority class.
- Apply threshold optimization on the validation set to improve F1-score.

This section does not present a finalized new method but rather outlines the experimental pipeline for the remaining project phase.

7. Ablation and Error Analysis Plan

Ablation Study Plan:

- Compare adapter fine-tuning vs. full fine-tuning of mBERT: measure AUC-ROC, F1, trainable parameter count, training time, and GPU memory usage.
- Vary adapter bottleneck dimension (64, 128, 256) to understand the effect of adapter capacity on cross-lingual performance.
- Compare with and without text cleaning preprocessing to assess its impact on transformer-based models.
- Evaluate the effect of maximum sequence length (64, 128, 256) on model performance and efficiency.

Error Analysis Plan:

- Generate confusion matrices per language to identify language-specific weaknesses.
- Analyze misclassified examples qualitatively: examine false positives (non-toxic flagged as toxic) and false negatives (toxic comments missed).

- Examine model confidence distributions for correct vs. incorrect predictions to identify overconfident errors.
- Compare error patterns across classical ML and transformer baselines to understand which types of toxic content each model struggles with.

8. Visualization and Interpretation Plan

The following visualizations are planned for the final report:

- Confusion matrices for each model and each language in the test set.
- ROC curves and precision-recall curves comparing all baselines and the proposed adapter method.
- Per-language AUC-ROC bar chart to visualize cross-lingual transfer effectiveness.
- Training loss and validation AUC curves for deep learning models (LSTM, mBERT, mBERT+adapter).
- Parameter count and inference time comparison bar chart across all methods.
- t-SNE visualization of mBERT hidden representations colored by language and toxicity label to examine cross-lingual representation alignment.
- Feature importance analysis for TF-IDF baselines using top weighted features for the toxic class.

9. GitHub and Reproducibility Status

Repository: <https://github.com/yasnrk/multilingual-toxic-comment-classification>

Current repository structure:

```
multilingual-toxic-comment-classification/
.gitignore
README.md
requirements.txt
src/
  preprocess.py      Data loading and text cleaning
  eda.py             Exploratory data analysis
  train_baselines.py TF-IDF baselines (LR, NB, SVM)
  train_transformer.py mBERT and adapter training
  evaluate.py        Metrics and visualization
results/            Experiment outputs (CSV/JSON)
figures/            Generated plots
data/               (gitignored)
models/             (gitignored)
```

The repository contains a complete Python-based project structure. All source code is organized under the `src/` directory as standalone `.py` scripts. The `README` includes setup instructions, dataset download steps from Kaggle, and usage examples for each script. A `requirements.txt` file lists all dependencies (scikit-learn, pandas, numpy, torch, transformers, matplotlib, seaborn). The baseline training script (`train_baselines.py`) and EDA script (`eda.py`) are fully functional and have produced the results reported in this document. The transformer training script is written and ready for GPU execution. Consistent commit history will be maintained going forward.

10. Current Challenges and Next Steps

Current Challenges:

- Class imbalance: Only ~10% of training comments are toxic, leading to low recall. Weighted loss and threshold tuning will be explored.
- GPU resources: Full fine-tuning of mBERT requires significant GPU memory. Google Colab Pro or Kaggle GPU will be used. Adapter-based training is expected to alleviate this.
- Cross-lingual evaluation: TF-IDF baselines are language-specific and cannot be directly evaluated on multilingual test data. Separate TF-IDF models or translation-based approaches may be needed for fair comparison.

- Commit history: The repository needs to be populated with code that has been developed locally. This will be addressed immediately.

Next Steps (Remaining Weeks):

- Week 1: Complete LSTM and mBERT full fine-tuning training runs. Obtain baseline results on multilingual test set.
- Week 2: Implement adapter-based mBERT fine-tuning. Run adapter experiments with different bottleneck sizes.
- Week 3: Conduct ablation study and error analysis. Generate all visualizations.
- Week 4: Write the final report. Finalize GitHub repository with full documentation and reproducibility instructions.

The project is on track. The classical baselines show reasonable performance, the dataset is prepared, and the literature review supports the technical direction. The main remaining work is completing the deep learning experiments and the proposed adapter-based method.